

ECN and L4S across the SCREAMv2 + 5G RAN stack

What is supported in the code, what the standards say, and the uplink question

A code- and standards-grounded research note for the GstreamerExp testbed. Every code claim links to the exact file and line on GitHub; standards claims cite an RFC / 3GPP TS / URL. Claims read from code are **measured**; design intent is marked **inferred**.

Bottom line (the three questions)

- 1. GStreamer SCREAMv2 supports ECN** fully in the C++ library (ECT marking, CE read, RFC 8888 feedback, and both a classic and an L4S/scalable response). The Rust *gstscream* plugin wires ECN on the *receiver* only; sender ECT-marking is a TODO — ECN on the wire is set instead by the `udp_ecn_diff.txt` patch to GStreamer's `multiudpsink`.
- 2. srsRAN does not mark ECN.** The only ECN in the tree is an unimplemented 3GPP Rel-18 E1AP signalling stub. The user plane is pass-through, so it *preserves* a pre-set ECN field but never *sets* one. Mainline OAI is the same.
- 3. Uplink ECN marking is standardized and already demonstrated — it is just missing from mainline open stacks.** 3GPP Rel-18 marks at the **gNB** (and UPF) for **both directions**, and **L4Span** (Princeton, CoNEXT) implements it on **srsRAN** for uplink and downlink. So the first-hop uplink case is achievable today; the gap is implementing it in mainline srsRAN / OAI, not the standard.

1 Fundamentals

ECN (Explicit Congestion Notification) lets a router signal congestion by *marking* a packet instead of dropping it. The signal lives in the low 2 bits of the IP header's traffic-class byte:

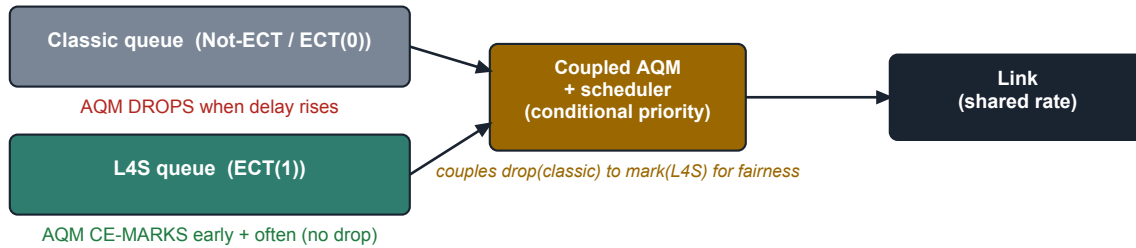
Codepoint	Bits	Meaning
Not-ECT	00	Sender is not ECN-capable; routers can only drop.
ECT(0)	10	ECN-capable, classic.
ECT(1)	01	ECN-capable, the L4S identifier.
CE	11	Congestion Experienced: a router marked this packet.

L4S (Low Latency, Low Loss, Scalable throughput) builds on ECN. Senders mark their packets **ECT(1)**; an L4S-aware bottleneck puts them in a separate low-latency queue and **marks CE early and often, instead of dropping**, as soon as queue delay starts to rise. A *scalable* sender (DCTCP-style) reacts in proportion to how often it is marked, so it sits at a shallow, stable queue.

Correcting one common picture

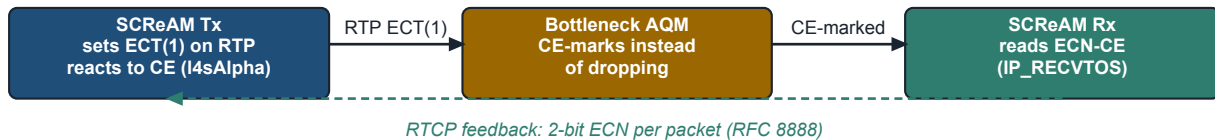
L4S is **not** “two queues where marked traffic is sent before regular traffic.” The L4S queue gets *conditional* priority for low latency, but a **Coupled AQM** ties its marking rate to the classic queue's drop rate so the two stay fair over time (RFC 9332). The latency win comes from the sender reacting to *early, frequent CE marks*, not from marked packets jumping the line.

Figure 3 — L4S DualQ: not 'marked traffic first', but early CE-mark instead of drop



SCReAM (RFC 8298) is Ericsson's self-clocked congestion control for real-time RTP media. It adapts the media bitrate to delay, loss, and ECN. With L4S it uses the scalable response. The receiver echoes the 2-bit ECN of every packet back to the sender in an **RFC 8888** RTCP feedback report, which is how a CE mark reaches the encoder.

Figure 1 — The SCReAM ECN control loop



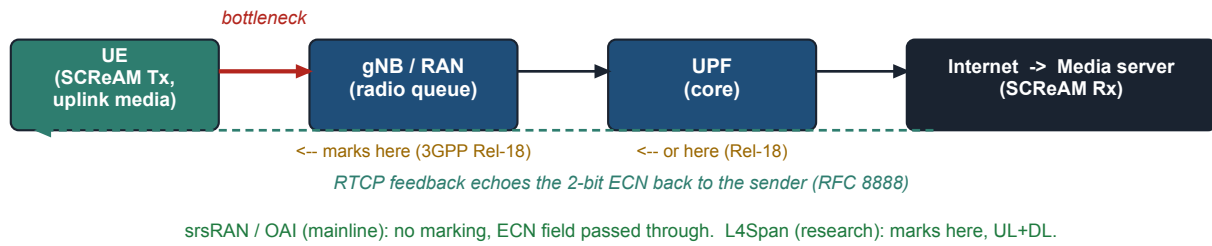
2 Problem context

Interactive media (cloud gaming, VR/XR, conversational video) needs low latency *under load*. When a link fills, classic congestion control only learns about it from loss or growing delay, both of which arrive late and hurt responsiveness. ECN/L4S gives the sender an *early* congestion signal, so it can trim its rate before the queue grows.

The cellular link is usually the bottleneck, and it is **asymmetric**:

- **Downlink** (server → UE) is the conventional case. Cloud-gaming frames flow down; the bottleneck and the marking both sit on the network side. This is what most L4S work and the T-Mobile deployment target.
- **Uplink** (UE → server) is your case: the bottleneck is the **first hop**, the UE's own radio transmission. The packets that need marking are leaving the UE, so the marking has to happen at the UE or at the gNB's uplink buffer. This is the harder, less-served direction.

Figure 2 — Where ECN-CE marking happens in the 5G path (and the uplink case)



3 Findings: does the stack support ECN?

3.1 GStreamer SCREAMv2 — yes, with a sender-plugin gap

The SCReAM C++ library (github.com/ericssonresearch/scream) does the full ECN loop (measured):

- **Sender sets ECT** on the socket with `IP_TOS / IPV6_TCLASS` — `scream_sender.cpp:552`.
- **Receiver reads CE** via `IP_RECVTOS + recvmsg cmsg` — `scream_receiver.cpp:288`.
- **Feedback** carries the 2-bit ECN per RFC 8888 — `ScreamRx.cpp:198`.
- **Controller reacts two ways** — classic (cut the window by `kEcncBeta=0.9`, `ScreamV2Tx.h:33`) and **scalable / L4S** (a DCTCP-like `l4sAlpha` from the fraction of marked bytes, `ScreamV2Tx.cpp:737`). It can even synthesise a “virtual” CE from queue delay when the network does not mark.

The Rust `gststream` GStreamer plugin only wires part of this:

Stage	C++ library	gststream plugin
Sender sets ECT(1)	Yes (socket option)	No — TODO at <code>sender.rs:71</code> ; set via the <code>udp_ecn_diff.txt</code> patch to <code>multiudpsink</code>
Receiver reads CE	Yes	Yes, behind the ecn-enabled feature (<code>screamrx/ecn.rs</code>)
RFC 8888 feedback	Yes	Yes
L4S / classic response	Yes	Yes (in the wrapped library)

The GStreamer ECN patch (!2717)

Sender-side ECN in GStreamer comes from an upstream merge request, [GStreamer !2717](#) (“udp: Add support for ECN to udpsink and udpsrc”, BBC R&D, 2022). It adds a `set-ecn` property to the UDP elements and a new core `GstNetEcnMeta` buffer metadata, so a pipeline can stamp ECT(1) on outgoing packets and read ECN-CE on incoming ones. The merge request is still open (last updated 2023), so the feature is not yet in released GStreamer; `gststream` ships it as `udp_ecn_diff.txt`, a 2024 re-base of !2717 applied to a from-source GStreamer build.

To actually run SCREAM-L4S with ECN in GStreamer

Build a patched GStreamer (`udp_ecn_diff.txt` adds a `set-ecn` property to `multiudpsink`), build `gststream` with `ECN_ENABLED=1`, select ECT(1) for L4S mode on the sender, and confirm the receiver prints `ecn-enabled`. Steps are in [README.md](#).

3.2 srsRAN / OAI — no active marking

In `srsRAN release_25_10`, the string `ecn` appears **only** in the E1AP ASN.1 stub (`e1ap_ies.cpp`): the 3GPP Rel-18 “ECN marking / congestion report” information element is *defined but not implemented*. There is no queue-occupancy-based CE-marking anywhere in SDAP, PDCP, RLC, MAC, or the scheduler (measured: a search for active marking returns nothing). The only socket-level marking is *control-plane* DSCP (`udp_network_gateway_impl.cpp:480`).

The user plane only forwards. The GTP-U uplink path adds the GTP-U header and a QFI extension and never touches the inner IP packet (`gtpu_tunnel_ngu_tx_impl.h:56`), so a pre-set ECN field is **preserved end to end** (measured) but `srsRAN` never *sets* one. Mainline `OpenAirInterface` is the same: no user-plane L4S marking. The `srsRAN` Project ships only the gNB (no NR UE), so the UE uplink path cannot be read from its source.

3.3 The research bridge: L4Span on srsRAN

L4Span (Princeton PAWS, ACM CoNEXT) — the answer to the uplink question

L4Span implements L4S congestion signalling on **srsRAN** (~2,000 lines of C++). It predicts the RAN's queue occupancy at millisecond timescales and performs ECN-CE marking for both low-latency and classic flows, with explicit **uplink and downlink** marking (code under [lib/mark](#)). It stays 3GPP / O-RAN compliant and reports up to ~98% lower one-way delay. This is a working precedent for exactly your setup: SCSReAM-L4S over srsRAN with uplink marking.

arXiv 2510.02682 · github.com/PrincetonUniversity/L4Span · CoNEXT

	Set ECT (sender)	Read / react to CE	CE-mark in network	Uplink marking
SCREAMv2 (C++ lib)	Yes	Yes	n/a (endpoint)	n/a
gstscram plugin	Via patched udpsink	Yes (receiver)	n/a	n/a
srsRAN mainline	No	No	No (E1AP stub only)	No (preserves field)
OAI mainline	No	No	No	No
L4Span (research, on srsRAN)	n/a	n/a	Yes	Yes

4 The uplink question and the standards

- **3GPP Rel-18** (TS 23.501, L4S support, clause 5.37) standardises ECN marking for L4S at the **NG-RAN (gNB)** and/or the **UPF**, selectable by the operator. The gNB marks based on radio congestion, and the feature covers **both uplink and downlink**. So uplink first-hop marking is specified, not a gap in the standard. (*sourced via secondary summaries of the TS; pin the exact clause text before quoting.*)
- **Open implementations lag the standard.** srsRAN and OAI mainline do not implement it; L4Span does (research).
- **T-Mobile** announced production L4S on its 5G-Advanced network (July 2025), the first US carrier to do so at scale, for apps like NVIDIA GeForce NOW, FaceTime, XR, and remote driving. L4S is ECN-based; the public materials emphasise the apps and do not break out uplink vs downlink.

t-mobile.com/news/network/unlock-l4s-5g-advanced

Net answer to “do UE stacks support uplink ECN?”

Standardised: **yes** (Rel-18, gNB/UPF, UL+DL). Mainline open RAN (srsRAN/OAI): **no**. Demonstrated on srsRAN in research: **yes** (L4Span). Your premise — congestion at the uplink first hop — is exactly what gNB-side Rel-18 marking targets; the work is porting/enabling it, not inventing it.

5 Practical options for the testbed

- **Media side:** run SCREAM in **L4S mode** (ECT(1)) with the patched GStreamer; the sender already reacts to CE with the scalable `L4sAlpha` response.
- **RAN marking, three paths:** (a) port/use **L4Span** on srsRAN for real UL+DL CE-marking (closest to your goal); (b) for a controlled experiment, put an **L4S AQM** (`DualPI2` / `fq_codel` with L4S) on a Linux bottleneck

in the path and let srsRAN pass the ECN field through (which it does); (c) rely on SCReAM's **virtual-CE** (queue-delay) mode where no network marking exists — useful as a fallback but not a true L4S signal.

- **Verify pass-through** first: confirm a CE set upstream survives the srsRAN bearer (the GTP-U path does not touch the inner IP header, so it should).

References

Public code repositories (open access)

- **SCReAM / SCREAMv2** + the *gststream* GStreamer plugin — github.com/EricssonResearch/scream
- **srsRAN Project** (open-source 5G gNB / CU / DU) — github.com/srsran/srsRAN_Project
- **OpenAirInterface 5G** (OAI) — gitlab.eurecom.fr/oai/openairinterface5g
- **L4Span** (L4S ECN marking on srsRAN, research prototype) — github.com/PrincetonUniversity/L4Span

Standards and sources

- RFC 8298 — SCReAM (self-clocked rate adaptation for multimedia). www.rfc-editor.org/rfc/rfc8298
- RFC 8888 — RTCP congestion-control feedback (echoes the 2-bit ECN). www.rfc-editor.org/rfc/rfc8888
- RFC 9330 / 9331 / 9332 — L4S architecture, ECN protocol (ECT(1)), DualQ Coupled AQM.
- RFC 3168 — ECN codepoints in the IP header.
- 3GPP TS 23.501 (Rel-18), clause 5.37 — L4S support; ECN marking at NG-RAN and/or UPF, UL+DL.
- L4Span (paper) — Princeton PAWS, ACM CoNEXT. arxiv.org/abs/2510.02682
- T-Mobile — L4S on 5G-Advanced (Jul 2025). www.t-mobile.com/news/network/unlock-l4s-5g-advanced

Code files cited (click to open the exact file on GitHub)

- SCReAM (master): [scream_sender.cpp](#), [scream_receiver.cpp](#), [ScreamRx.cpp](#), [ScreamV2Tx.cpp](#), [gststream/sender.rs](#), [gststream/screamrx/ecn.rs](#)
- srsRAN @ release_25_10 (d2f4b70): [e1ap_ies.cpp](#), [gtpu_tunnel_ngu_tx_impl.h:56](#), [udp_network_gateway_impl.cpp:480](#)